

CollegeBot

Complete Code Guide & Viva Q&A

A beginner-friendly walkthrough of every file in the project

RAG Chatbot for College FAQ - built with Python

Live app: <https://chatbot-mem.streamlit.app/>

GitHub: <https://github.com/dhyey-gosa/college-chatbot>

How to use this guide: Read Part 1 to understand the big picture. Part 2 explains each file line-by-line in plain English. Part 3 is a glossary of every technical term. Part 4 is the Q&A bank for when ma'am asks questions. Part 5 is the live demo script - follow it click-by-click during the presentation.

Part 1 - What We Built (The Big Picture)

CollegeBot is a website chatbot that answers questions about a college (fees, library timings, hostel rules, admission dates, etc.). Unlike a normal chatbot that only knows what it was trained on, CollegeBot first **searches its own notes** (a knowledge base of college facts), then writes an answer **using only those notes**. This technique is called **RAG - Retrieval Augmented Generation**.

1.1 One-sentence summary for ma'am

"CollegeBot is a Retrieval-Augmented Generation chatbot: it retrieves the most relevant chunks from a college knowledge base stored in MySQL (table `chunks`), then generates an answer with a large language model (via the Groq API), and logs every Q&A into MySQL with a Streamlit web UI."

1.2 How a question flows through the system

```

User types a question in the browser
      |
      v
[app.py] Streamlit UI receives the text box input
      |
      v
[rag.py] --- RETRIEVE ---
        Embed the question into numbers, cosine-rank MySQL `chunks`,
        grab the top 3 most similar text rows
      |
      v
[rag.py] --- AUGMENT ---
        Paste those 3 chunks into a prompt as CONTEXT
      |
      v
[rag.py] --- GENERATE ---
        Send prompt to Groq LLM (openai/gpt-oss-120b),
        get back a plain-English answer
      |
      v
[database.py] Save question, answer, sources, time
              into MySQL chat_logs table (or local JSON)
      |
      v
[app.py] Show answer on screen + thumbs up/down buttons
  
```

1.3 Tech stack (what each tool does)

Tool	Role in this project
Python 3.11	The programming language everything is written in.
Streamlit	Turns Python into a web UI (chat page + admin page) with almost no HTML/CSS.
LangChain	Glue library that connects LLMs, prompts, and vector databases.
Groq API	Free/fast cloud service that runs the LLM and returns answers.
MySQL (4 tables)	chunks (vectors + text), chat_logs, documents, feedback. JSON file fallback if MySQL is off.
sentence-transformers	Local model (all-MiniLM-L6-v2) that converts text into embedding vectors.

pymysql	Python driver that talks to MySQL.
PyPDF / TextLoader	Reads PDF and TXT files so we can ingest their content.
python-dotenv	Loads secret keys from a .env file into environment variables.
GitHub	Hosts the source code; Streamlit Cloud pulls from it to deploy.
Streamlit Community Cloud	Free hosting that runs streamlit run app.py and gives a public URL.

1.4 Project file map

File	Plain-English job
app.py	The whole website: Chat page and password-protected Admin page (tables).
rag.py	The brain. Does retrieve (MySQL cosine search), augment, generate.
ingest.py	Reads a file, splits it into chunks, embeds them, stores in MySQL chunks table.
database.py	All MySQL code: documents, chat_logs, feedback, chunks. Falls back to local_data.json.
config.py	Loads secrets (.env / Streamlit secrets) and model names in one place.
knowledge_base.txt	Starter facts about Sunrise College (fees, library, hostel, etc.).
requirements.txt	Exact list of Python packages to install (pip install -r requirements.txt).
render.yaml	Optional Render.com deploy config (we actually used Streamlit Cloud).
.env / .env.example	Secret keys file. Real .env is NEVER pushed to GitHub.
.gitignore	Tells git which files to ignore (.env, venv, caches...).
local_data.json	Created at runtime when MySQL is OFF - same 4 collections as the tables.

Part 2 - File-by-File Code Walkthrough

Read this part top to bottom before the demo. Every important block of code is shown with a plain-English explanation of what it means.

2.1 config.py - Settings & Secrets (52 lines)

What it does: Reads all secret keys and settings from one place so keys are never hard-coded inside app logic. Locally it reads a .env file; on Streamlit Cloud it reads App Settings -> Secrets. Same code works in both places.

Key code:

```
import os
from dotenv import load_dotenv
load_dotenv() # reads .env file into environment variables

def _secret(name: str, default: str = "") -> str:
    val = os.getenv(name, "") # 1) try environment variable
    if val:
        return val
    try:
        import streamlit as st
        if hasattr(st, "secrets") and name in st.secrets:
            return str(st.secrets[name]) # 2) else try Streamlit secrets
    except Exception:
        pass
    return default

GROQ_API_KEY = _secret("GROQ_API_KEY") # key for the LLM
LLM_MODEL = "openai/gpt-oss-120b" # which Groq model to use
EMBED_MODEL = "all-MiniLM-L6-v2" # local embedding model
ADMIN_PASSWORD = _secret("ADMIN_PASSWORD", "college123")
```

- **load_dotenv()** - loads key=value pairs from the hidden .env file into the process environment.
- **_secret(name)** - tries env var first, then Streamlit secrets, then a default. Dual-source lookup so local + cloud both work.
- **GROQ_API_KEY** - the password that lets our app call Groq's free LLM API. Never commit the real key.
- **LLM_MODEL** - which Groq model to call. We use openai/gpt-oss-120b (available on our key).
- **EMBED_MODEL** - name of the sentence-transformers model. It runs on our CPU, no API key needed.
- **ADMIN_PASSWORD** - gate for the Admin page. Default demo value is college123.
- **DB_CONFIG** - MySQL connection details. Empty DB_HOST means local JSON mode (no MySQL).

2.2 rag.py - The RAG Brain (MySQL vector search)

What it does: Implements the full Retrieval Augmented Generation pipeline. Retrieval reads the MySQL **chunks** table (or local_data.json), computes cosine similarity against the question embedding, and keeps the top 3. If the LLM is a student writing an *open-book* exam, those 3 rows are the book pages.

Core idea in code:

```
def retrieve(question, k=3):
    rows = get_all_chunks() # FROM chunks (MySQL or JSON)
    q_vec = embed_text(question) # sentence-transformers, 384 floats
```

```

scored = [(_cosine(q_vec, r["embedding"]), r) for r in rows]
scored.sort(reverse=True)          # highest similarity first
return top k rows

def ask(question):
    hits = retrieve(question, k=3)          # 1) RETRIEVE
    context = "\n\n".join(h["content"] for h in hits) # 2) AUGMENT
    messages = [SystemMessage(SYSTEM_PROMPT.format(context=context)),
                HumanMessage(question)]
    answer = _get_llm().invoke(messages).content # 3) GENERATE
    return answer, sources, elapsed_seconds

```

The system prompt (personality + rules):

You are CollegeBot, a helpful assistant for college students.
 Answer ONLY using the context below. If the answer is not in the context,
 say politely: "I don't have that in my knowledge base yet."
 Keep answers short and friendly (max 5 sentences).

```

CONTEXT:
{context}

```

- **get_all_chunks()** - loads every row from MySQL table `chunks` (or local JSON); embedding is a JSON array of floats.
- **_cosine(a, b)** - similarity score between question vector and each chunk vector; we sort descending and keep top 3.
- **k=3** - we only pass 3 chunks to the LLM. Fewer chunks = faster + cheaper + less noise.
- **SYSTEM_PROMPT** - the LLM's job description. Critical line: *Answer ONLY using the context* - this is what stops hallucination.
- **HumanMessage / SystemMessage** - LangChain's way of building the chat messages sent to Groq.
- **_get_llm() / _get_embeddings()** - lazy singletons: models are created once on first use, not at import time.
- **temperature=0.2** - low creativity so answers stay factual and consistent.
- **embed_text()** - runs sentence-transformers on CPU and returns a normalized 384-float vector.
- **Return value** - a tuple (answer_text, sources_text, seconds_taken) so the UI can show timing + source file.

2.3 ingest.py - Teaching the Bot New Knowledge

What it does: Turns a raw PDF/TXT into searchable vector rows in MySQL. This is the *indexing* step of RAG - done once when we add a document, not on every chat message.

```

def ingest_pdf(path):
    # 1) LOAD - read raw text
    if path.lower().endswith(".pdf"):
        raw_docs = PyPDFLoader(path).load()
    else:
        raw_docs = TextLoader(path, encoding="utf-8").load()

    # 2) SPLIT - ~800 char chunks, 100 char overlap
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=800, chunk_overlap=100)
    pieces = splitter.split_documents(raw_docs)

    # 3) EMBED each piece  4) STORE into MySQL `chunks`
    items = [{"content": d.page_content,
              "source": d.metadata["source"]},

```

```
"embedding": embed_text(d.page_content)} for d in pieces]
return add_chunks(items) # INSERT into chunks table
```

- **LOAD** - PyPDFLoader extracts text page-by-page from a PDF; TextLoader reads a plain .txt file.
- **SPLIT (chunking)** - models work better on small pieces. 800 characters is about one short paragraph/section.
- **chunk_overlap=100** - consecutive chunks share 100 characters so a sentence cut in half is not lost.
- **embed_text()** - each chunk becomes a 384-float vector via all-MiniLM-L6-v2 (local CPU).
- **add_chunks()** - database.py INSERTs content + source + JSON embedding into MySQL `chunks` (or appends to local_data.json).
- **CLI mode** - python ingest.py knowledge_base.txt works from the terminal for batch loading.

Analogy: ingest.py is like filing each page of a textbook into a MySQL cabinet with a numeric fingerprint (embedding) so retrieval can find the right page later.

2.4 database.py - MySQL (4 tables) + JSON Fallback

What it does: Every database operation lives here. Four tables: **chunks** (knowledge text + embedding vector - replaces Chroma), **documents** (what we ingested), **chat_logs** (every Q&A), **feedback** (thumbs up/down). If MySQL is not configured or fails, everything writes to local_data.json so the demo never crashes.

```
CREATE TABLE chunks (
    id INT AUTO_INCREMENT PRIMARY KEY,
    content TEXT,           -- the raw text chunk
    source VARCHAR(255),    -- which file it came from
    embedding LONGTEXT,     -- JSON array of 384 floats
    added_at DATETIME
);

def add_chunks(items):
    # MySQL: INSERT content, source, JSON.stringify(embedding)
    # fallback: append dicts into local_data.json["chunks"]

def get_all_chunks():
    # SELECT ... FROM chunks -> list of dicts with embedding as list[float]

def log_chat(...):
    # INSERT INTO chat_logs ...; return LAST_INSERT_ID()
```

- **chunks table** - the vector store itself. embedding column holds a JSON array like [0.02, -0.11, ...].
- **add_chunks / get_all_chunks / count_chunks** - write and read the knowledge base for RAG.
- **_connect()** - returns a pymysql connection or None. None means "use local JSON instead".
- **setup_tables()** - CREATE TABLE IF NOT EXISTS for all 4 tables; run once at startup from app.py.
- **DictCursor** - makes each row a Python dict so we can do row["question"] instead of row[0].
- **log_chat()** - INSERT one conversation, return its id (used later to attach feedback).
- **LAST_INSERT_ID()** - MySQL function that returns the auto-increment id of the row we just inserted.
- **Parameterized queries (%s)** - user text is never pasted into SQL strings, which prevents SQL injection.
- **get_chat_logs / get_documents / get_feedback_logs** - SELECT helpers that feed the Admin tables (feedback LEFT JOINS the question).

- **local_data.json fallback** - same 4 keys as the tables so the app works with MySQL off.

2.5 app.py - The Whole User Interface

What it does: Builds the website with Streamlit. Sidebar has a radio button to switch between two pages: Chat (students) and Admin (password-protected upload + all data shown as tabular dataframes).

Page setup + sidebar:

```
import streamlit as st
st.set_page_config(page_title="CollegeBot", page_icon="[College]", layout="wide")

db_status = setup_tables() # create MySQL tables once, show status

with st.sidebar:
    st.title("[College] CollegeBot")
    st.text(f"Database: {db_status}")
    page = st.radio("Go to", ["[Chat] Chat", "[Admin] Admin"])
```

Chat page - the main loop:

```
if page == "[Chat] Chat":
    if "messages" not in st.session_state:
        st.session_state.messages = [] # memory between reruns

    for m in st.session_state.messages: # redraw old messages
        with st.chat_message(m["role"]):
            st.markdown(m["text"])

    user_q = st.chat_input("Type your question here...")
    if user_q:
        st.session_state.messages.append({"role": "user", "text": user_q})
        answer, sources, secs = ask(user_q) # <-- calls rag.py
        st.session_state.messages.append(
            {"role": "assistant", "text": answer})
        log_chat(user_q, answer, sources, secs) # <-- saves to MySQL
        # then show thumbs up / thumbs down buttons
```

- **st.session_state** - Streamlit's memory. Streamlit re-runs the whole script on every click; session_state keeps chat history alive.
- **st.chat_message / st.chat_input** - built-in chat bubbles and the bottom text box.
- **ask(user_q)** - one call into rag.py returns (answer, sources, seconds).
- **try/except around ask()** - if the API key is missing or Groq errors, we show a warning instead of a red crash screen.
- **log_chat + log_feedback** - persistence layer (MySQL or JSON).

Admin page (password gate):

```
if not st.session_state.admin_ok:
    pw = st.text_input("Enter admin password", type="password")
    if st.button("Login"):
        if pw == ADMIN_PASSWORD:
            st.session_state.admin_ok = True
            st.rerun()
        else:
            st.error("Wrong password")
    st.stop() # nothing below runs until login succeeds
```

- **type="password"** - masks the input as dots.
- **st.stop()** - hard stop: the rest of the Admin page is invisible until the password is correct.

- **File uploader** - saves the uploaded PDF to uploads/, calls ingest_pdf(), then log_document().
- **Auto-ingest at startup** - if count_chunks()==0 and knowledge_base.txt exists, load it once (fresh cloud deploys).

Admin data as TABLES (the part ma'am will like):

```
tab_chats, tab_docs, tab_fb, tab_chunks = st.tabs(
    ["Chat logs", "Documents", "Feedback", "Chunks"])

st.dataframe(rows, use_container_width=True) # proper table UI
# Feedback tab: LEFT JOIN feedback with chat_logs to show the question
# Chunks tab: id, source, chars, embedding dim, text preview
# (full 384-float vectors stay in MySQL, not dumped into the UI)
```

- **st.tabs** - four sections on one Admin page without scrolling forever.
- **st.dataframe** - sortable, scrollable table (not expanders). Columns: ID, question, answer, sources, seconds, timestamp.
- **Metrics row** - chunks in DB, documents ingested, chats logged.
- **Feedback join** - get_feedback_logs() LEFT JOINS so each rating shows which question it was for.

2.6 knowledge_base.txt - The Demo Data (39 lines)

Plain text file with facts about a fictional **Sunrise College of Engineering**: admissions, fees (tuition 85,000 / hostel 45,000), library (8 AM-8 PM), hostel rules, exams (75% attendance), and contact info. This is the only knowledge the bot has until an admin uploads more PDFs. Every answer cites this file as its source.

2.7 requirements.txt - Exact Dependencies (19 lines)

```
--extra-index-url https://download.pytorch.org/whl/cpu
torch==2.14.0+cpu          # CPU-only torch (smaller, free-tier friendly)

streamlit==1.40.0          # web UI
langchain==0.3.7           # RAG framework
langchain-core==0.3.27     # must match langchain version (compat)
langchain-groq==0.2.1      # Groq LLM wrapper
sentence-transformers==3.3.1 # local embeddings
langchain-huggingface==0.1.2 # HF embeddings integration
pypdf==5.1.0               # PDF reader
pymysql==1.1.1             # MySQL driver (vector store + logs)
python-dotenv==1.0.1       # loads .env
gunicorn==23.0.0           # production WSGI server (Render)
```

- Pinned versions (=) so the same code runs on our laptop and on the cloud.
- CPU torch avoids downloading the huge CUDA/GPU build - free hosts only have CPU.
- No chromadb - vectors live in MySQL `chunks` (JSON fallback).
- langchain-core must be compatible with langchain; a mismatch is the #1 install failure.

2.8 .env, .env.example, .gitignore - Secret Safety

.env holds the real GROQ key and admin password. **.env.example** is a safe template with placeholders, committed to GitHub so others know what to fill in. **.gitignore** excludes .env, venv/, __pycache__/, local_data.json, and uploads/ so secrets and junk never hit the repo. Vectors are not committed - they live in MySQL (or local_data.json) and are rebuilt by ingest / first-run auto-ingest.

Part 3 - Glossary (Every Technical Term, Plain English)

- **RAG (Retrieval Augmented Generation)** - Search your own documents first, then let the LLM write an answer using those documents as open-book notes. Reduces hallucination.
- **LLM (Large Language Model)** - A big neural network trained to predict text (ChatGPT-style). Here we call one through Groq's API instead of hosting it ourselves.
- **Embedding** - A list of numbers (vector) that represents the meaning of a sentence. Similar meaning = nearby points in that number space. Our model outputs 384 numbers per chunk.
- **Vector store (MySQL chunks)** - Table `chunks` holds text + source + embedding (JSON array of floats). Retrieval loads rows and ranks them by cosine similarity in Python.
- **Similarity search** - Given a query vector, score every stored vector by cosine similarity and return the top k. We use k=3.
- **Chunking** - Splitting a long document into ~800-character pieces so retrieval is precise and embeddings stay meaningful.
- **Prompt / System prompt** - The text sent to the LLM. System prompt sets rules (answer only from context); Human message is the user's question.
- **Temperature** - How creative/random the LLM is. 0 = very factual, 1 = wild. We use 0.2 for a student-helper tone.
- **Hallucination** - When an LLM invents confident-sounding facts. Our guard: 'Answer ONLY using the context' + refuse if missing.
- **Streamlit rerun model** - Every interaction re-executes app.py from top to bottom. st.session_state is how we keep state between reruns.
- **session_state** - A dict owned by Streamlit for the current browser session - holds chat history and admin login flag.
- **Lazy initialization** - Create heavy objects (models) only when first needed. Our _get_llm() / _get_embeddings() use this pattern.
- **Environment variable / secrets** - Key-value settings outside the source code. Read via os.getenv or st.secrets so keys are not committed to git.
- **SQL injection** - Attack where user text becomes executable SQL. Prevented by parameterized queries with %s placeholders.
- **MySQL tables** - documents = files ingested; chat_logs = every Q&A; feedback = thumbs up/down linked by chat_id.
- **JSON fallback** - If MySQL is off, data goes to local_data.json with the same structure so the app still works offline.
- **PyPDFLoader** - LangChain loader that extracts text from each page of a PDF into Document objects.
- **RecursiveCharacterTextSplitter** - Splits text by characters first, then tries paragraph/newline boundaries so cuts land on natural breaks.
- **Groq** - Free API platform that serves open models at very high speed. We authenticate with GROQ_API_KEY.
- **Deployment** - Putting the app on a public server. We pushed to GitHub and Streamlit Community Cloud runs streamlit run app.py for us.
- **.gitignore** - List of paths git must never track (secrets, virtualenvs, caches).

- **Virtual environment (venv)** - A private Python install folder for this project so packages don't clash with other projects.

Part 4 - Viva / Presentation Q&A Bank

These are the questions ma'am is most likely to ask. Answers are written so you can read them almost verbatim. Practice the bolded ones first.

Concept questions

Q: What is RAG and why did you use it instead of fine-tuning?

A: RAG means Retrieval Augmented Generation. Instead of retraining the model, we fetch the 3 most relevant chunks from our college knowledge base and paste them into the prompt as context. The LLM then answers from that context. It is cheaper than fine-tuning, updates instantly when we change the notes, and lets us show the source of every answer.

Q: Explain the three steps of your RAG pipeline.

A: 1) RETRIEVE: embed the user question, load MySQL `chunks`, cosine-rank, keep top 3. 2)

AUGMENT: join those three chunks into one context string and place it inside a system prompt. 3)

GENERATE: send the prompt to the Groq LLM and return its answer, the source filenames, and the time taken.

Q: What is an embedding? Which model do you use?

A: An embedding is a fixed-length list of numbers that encodes meaning so similar sentences sit close together in vector space. We use the sentence-transformers model all-MiniLM-L6-v2, which runs locally on CPU and produces 384-dimensional vectors. It needs no API key.

Q: Why local embeddings instead of an API like OpenAI or Gemini embeddings?

A: Local embeddings are free, work offline, keep data on our machine, and avoid a second API key. The model is only about 80MB and downloads once to the Hugging Face cache.

Q: What is Chroma / where did it go?

A: We originally used Chroma as the vector DB. Now vectors live in the MySQL table `chunks` (content, source, embedding as JSON). Cosine similarity is computed in Python after loading rows. If MySQL is off, local_data.json stores the same structure.

Q: Why k=3 for retrieval? What happens if k is larger or smaller?

A: k=3 gives enough context for a short factual answer without flooding the prompt. Smaller k may miss the fact; larger k costs more tokens, can add noise, and may confuse the model. It is a tunable hyperparameter.

Q: How is the vector stored in MySQL?

A: The chunks.embedding column is LONGTEXT holding a JSON array of 384 floats produced by all-MiniLM-L6-v2. On query we parse the JSON, compute cosine similarity with the question embedding, and take the top 3 rows.

Q: How do you prevent the chatbot from hallucinating?

A: The system prompt says Answer ONLY using the context and to reply 'I don't have that in my knowledge base yet' if the answer is missing. Temperature is low (0.2). We also display the sources under each answer so users can verify.

Q: What is the role of LangChain in this project?

A: LangChain is the orchestration framework. It provides document loaders, text splitters, chat message types, and the ChatGroq client so we can wire retrieval and generation with a few lines instead of writing everything from scratch.

Q: What is Groq and which model are you calling?

A: Groq is a free high-speed API for open LLMs. We call model openai/gpt-oss-120b with temperature 0.2 through the langchain-groq ChatGroq class, authenticated by GROQ_API_KEY.

Q: How does Streamlit work under the hood?

A: Streamlit reruns the entire Python script whenever the user interacts (types, clicks). st.session_state preserves data between reruns - that is how our chat history and admin login survive.

Database questions**Q: What tables are in your MySQL database?**

A: Four tables: chunks (knowledge text + JSON embedding vector), documents (filename, chunks, added_at), chat_logs (question, answer, sources, seconds, asked_at), and feedback (chat_id, rating, given_at) linked to a chat row.

Q: Show how you insert a chat into MySQL.

A: log_chat() uses a parameterized INSERT: INSERT INTO chat_logs (question, answer, sources, seconds, asked_at) VALUES (%s,%s,%s,%s,%s), then SELECT LAST_INSERT_ID() to get the new row id for feedback.

Q: What if MySQL is down during the demo?

A: database.py detects a failed or missing connection and falls back to local_data.json with the same data shape. The sidebar shows 'MySQL OFF (local JSON mode)' instead of crashing.

Q: How do you avoid SQL injection?

A: Every query uses pymysql placeholders (%s) with values passed separately. User text is never concatenated into the SQL string.

Q: What is the use of feedback table?

A: It stores which chat got a thumbs up or down (chat_id + rating). Over time this shows which answers were helpful and can drive improvements.

Q: How would you show live chat logs to ma'am?

A: Open the Admin page, enter the password, and open the Chat logs tab - a sortable dataframe with ID, question, answer, sources, seconds, and timestamp. Under the hood it is SELECT ... ORDER BY id DESC LIMIT 50.

Architecture & deployment questions**Q: Walk me through what happens when a user asks a question.**

A: app.py captures the chat_input string, appends it to session_state, calls rag.py ask(). ask() loads all chunks from MySQL, embeds the question, cosine-ranks, builds the system+human messages with the top 3, invokes Groq, and returns answer/sources/seconds. app.py renders the bubble, logs the row in database.py, and shows feedback buttons.

Q: Where is the knowledge base loaded from? What if it is empty?

A: From the MySQL chunks table (or local_data.json if MySQL is off). On startup, if count_chunks() is 0 and knowledge_base.txt exists, app.py auto-ingests it once. On the Admin page you can upload any PDF, which is chunked and inserted the same way.

Q: How is the project deployed?

A: Source is on GitHub (dhyey-gosa/college-chatbot). Streamlit Community Cloud connects to that repo, installs requirements.txt, and runs streamlit run app.py. Secrets (GROQ_API_KEY, ADMIN_PASSWORD) are pasted into the app's Secrets settings as TOML. Live URL: <https://chatbot-mem.streamlit.app/>.

Q: Why didn't you hard-code the API key in the source?

A: Hard-coding keys in git leaks them to anyone with repo access. We use .env locally (gitignored) and Streamlit secrets in the cloud. config.py reads env first, then st.secrets.

Q: What is in requirements.txt and why pin versions?

A: It is the exact dependency list for pip install -r requirements.txt. Pinning (=) prevents 'works on my machine' bugs - Streamlit Cloud must install the same versions we tested.

Q: Explain chunk_size=800 and chunk_overlap=100.

A: Each piece of text stored is about 800 characters. Adjacent pieces share 100 characters so a sentence split across a boundary still appears fully in at least one chunk, improving recall.

Q: How would you scale this to 10,000 documents?

A: Add an ANN index path (or move vectors to a dedicated vector DB / MySQL 9 VECTOR type), index chunks by department metadata, use hybrid keyword+vector search, batch embeddings, and put the Groq call behind a queue. Also switch MySQL to a managed instance with connection pooling.

Q: What are the limitations of your current system?

A: Only top-3 chunks are used so multi-hop questions may be incomplete; answers are limited to the knowledge base (no web search); feedback is stored but not yet used for training; free hosting can sleep when idle; embeddings are English-focused.

Q: How do you evaluate answer quality?

A: Manual spot-checks against knowledge_base.txt, response time shown under each answer (e.g. 0.68s), source attribution for verifiability, and [Yes]/[No] feedback counts in the feedback table. A future step is an automated faithfulness score (does every claim appear in the retrieved chunks?).

Q: Is this secure?

A: Secrets stay out of git; Admin page uses a password gate; SQL uses parameterized queries; HTTPS is provided by the host. For production we would add proper auth (SSO), rate limiting, and server-side

session expiry - the current gate is demo-grade.

Code-specific questions (they may open a file)

Q: What does lazy loading mean in rag.py?

A: `_llm` and `_embeddings` start as `None`. `_get_llm()` and `_get_embeddings()` create the real objects only on the first call and reuse them after. Benefits: app starts faster, and a missing API key raises a clear `RuntimeError` instead of crashing at import time.

Q: Why store embeddings as JSON in MySQL instead of a separate vector DB?

A: For a college demo the corpus is tiny (a few dozen chunks). Loading rows and computing cosine in Python is simple, uses one database we already need for chat logs, and shows ma'am real SQL tables on the Admin page - no extra infrastructure.

Q: What does `st.stop()` do on the Admin page?

A: It halts the rest of the script for this rerun. Used after the password form so the upload UI and chat logs never render for a logged-out user.

Q: Why `try/except` around `ask()` in `app.py`?

A: So a missing key or API outage shows a friendly warning bubble instead of a full-page Streamlit exception. Keeps the demo alive when something external fails.

Q: Why is `.env` not committed while demo data can be rebuilt?

A: `.env` holds the live API key and must never be public. Knowledge-base vectors are not secret - we rebuild them with `python ingest.py knowledge_base.txt` or first-run `auto-ingest`. Chat history persists in MySQL when `DB_*` is configured.

Part 5 - Live Demo Script (Follow This Click-by-Click)

Open <https://chatbot-mem.streamlit.app/> in a browser **before** the class (first load can take ~30s while the app wakes up). Keep this tab ready.

Opening line (30 seconds)

"This is CollegeBot, a RAG-based chatbot for college FAQs. Instead of blindly trusting an LLM, it first retrieves the three most relevant lines from our knowledge base, then generates an answer from them, shows the source file, and logs everything into a database."

Demo steps

- > **Step 1.** Point at the sidebar: app name, Database status (MySQL OFF / connected), and the Chat/Admin switch.
- > **Step 2.** Ask: **What are the college fees?** - wait for the answer (usually under 1 second).
- > **Step 3.** Highlight: correct figures (85,000 / 45,000), the **sources: knowledge_base.txt** line, and the response time.
- > **Step 4.** Click **[Yes] Helpful** - explain this writes a row into the feedback table (or JSON fallback).
- > **Step 5.** Ask a second question from a different section, e.g. **What are the library timings?**
- > **Step 6.** Ask an out-of-scope question, e.g. **What is the stock price of Reliance?** - bot should refuse: not in knowledge base. This proves the hallucination guard.
- > **Step 7.** Switch to **[Admin] Admin**, password **college123**.
- > **Step 8.** Show the four Admin tabs as tables: Chat logs, Documents, Feedback, Chunks (st.dataframe).
- > **Step 9.** (Optional) Upload a small PDF and re-check the chunk count increases.
- > **Step 10.** Close with the architecture line from Part 1.2 and invite questions from Part 4.

Backup if the live site is slow

```
# run locally on the laptop
cd college-chatbot
venv\Scripts\activate
streamlit run app.py
# open http://localhost:8501
```

Local run uses the same code and the same knowledge base. Have the terminal pre-started as a fallback.

Likely last-minute questions (one-line answers)

Q: Is this using ChatGPT?

A: No. It calls a Groq-hosted open model (gpt-oss-120b) through an API, grounded by our RAG pipeline.

Q: Is the data going to Google/OpenAI?

A: Retrieval is fully local/MySQL: embeddings via sentence-transformers on our machine, ranking in Python/SQL. Only the final prompt (context + question) is sent to Groq to generate text.

Q: Can anyone become admin?

A: Only if they know ADMIN_PASSWORD. It is a demo gate; production would use real auth.

Q: What happens if you add a new PDF?

A: Admin upload -> ingest.py chunks and embeds it -> new rows appear in the MySQL chunks table -> the next question can retrieve from that PDF. Source name shows under the answer.

Q: Why is everything in one MySQL database?

A: One system for both operational data (chat_logs, documents, feedback) and the vector store (chunks). Admin page shows all four as tables; JSON file is only the offline fallback.

Q: What is the accuracy?

A: Within the knowledge base, answers match the source facts exactly (verified live: fee question returned 85,000 / 45,000 / 15 Aug 2026). Outside the KB it refuses instead of guessing.

End of guide. Source code: GitHub [dhyey-gosa/college-chatbot](https://github.com/dhyey-gosa/college-chatbot). Live app: <https://chatbot-mem.streamlit.app/>. Admin password is in .env / Streamlit secrets (default demo: college123).